Lecture 26.07: Linear Regression with Ordinary Least Squares (OLS) — Closed-Form Normal Equation

📌 Executive Summary

Analytical closed-form solution for linear regression via the Normal Equation, matrix calculus derivation, NumPy implementation using np.linalg.inv, and zero-iteration convergence.

🔬 Mathematical Formulation & Theory

1. Why Search Iteratively When an Exact Formula Exists?

Gradient Descent is an iterative approximation algorithm requiring hundreds of epochs and careful hyperparameter tuning of the learning rate α. For Linear Regression, the cost function is a strictly convex quadratic equation. By setting the vector derivative directly to zero, we can solve for the optimal parameter vector $\boldsymbol{\theta}^*$ in a single analytical step!

2. Matrix Formulation & Derivation

Augment the design matrix $X \in \mathbb{R}^{m \times d}$ with an initial column of ones to represent the intercept term:

$$X_b = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \dots & x_d^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times (d+1)}, \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \in \mathbb{R}^{d+1}$$

The sum of squared residuals in vector notation is:

$$J(\boldsymbol{\theta}) = \|X_b\boldsymbol{\theta} - \mathbf{y}\|_2^2 = (X_b\boldsymbol{\theta} - \mathbf{y})^T(X_b\boldsymbol{\theta} - \mathbf{y})$$

Expanding the matrix product:

$$J(\boldsymbol{\theta}) = \boldsymbol{\theta}^T X_b^T X_b \boldsymbol{\theta} - 2\mathbf{y}^T X_b \boldsymbol{\theta} + \mathbf{y}^T \mathbf{y}$$

Taking the vector derivative with respect to $\boldsymbol{\theta}$:

\nabla_\boldsymbol{\theta} J = 2 X_b^T X_b \boldsymbol{\theta} - 2 X_b^T \mathbf{y}

Setting the gradient to zero (\\nabla_\boldsymbol{\theta} J = \mathbf{0}):

$$2X_b^T X_b \boldsymbol{\theta} = 2X_b^T \mathbf{y} \implies X_b^T X_b \boldsymbol{\theta} = X_b^T \mathbf{y}$$

Assuming the Gram matrix $X_b^T X_b$ is non-singular (invertible), multiplying both sides by $(X_b^T X_b)^{-1}$ yields the celebrated **Normal Equation**:

$$\boldsymbol{\theta} = (X_b^T X_b)^{-1} X_b^T \mathbf{y}$$

3. OOP Implementation

```
class LinearRegressionOLS:
    def __init__(self):
        self.bias = None
        self.weights = None

    def fit(self, X, y):
        m, n = X.shape
        X_b = np.c_[np.ones((m, 1)), X]
        theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
        self.bias = theta[0]
        self.weights = theta[1:]

    def predict(self, X):
        return self.bias + np.dot(X, self.weights)
```

💻 Python & NumPy Implementation

```
import numpy as np

class LinearRegressionOLS:
```

```
    """Analytical Linear Regression via Closed-Form Normal Equation."""
    def __init__(self):
        self.bias = None
        self.weights = None

    def fit(self, X, y):
        m, n = X.shape
        # Augment bias column [1, X]
        X_b = np.c_[np.ones((m, 1)), X]

        # Closed-form Normal Equation: theta = (X_b^T X_b)^(-1) X_b^T y
        theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

        self.bias = theta[0]
        self.weights = theta[1:]

    def predict(self, X):
        return self.bias + np.dot(X, self.weights)

# Test with Lecture Example
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

ols = LinearRegressionOLS()
ols.fit(X, y)
print("OLS Bias:       ", ols.bias)
print("OLS Weights:    ", ols.weights)
print("OLS Predictions:", ols.predict(X))
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.